

Sistema adaptativo de decisión ante fraude transaccional

Curso: Planificación y Toma de Decisiones en IA — UTEC **Dataset:** IEEE-CIS Fraud Detection · **Semilla:** 42 · **Retraso de etiqueta:** L = 30 días

Origen de los datos de esta corrida. Los resultados se obtuvieron sobre un **dataset sustituto sintético** que replica el esquema de IEEE-CIS (394 columnas), su eje temporal (182 días, ~3 200 tx/día), su prevalencia (3,52 %) y su cobertura de identidad (24 %), con drift inyectado de parámetros conocidos. IEEE-CIS exige un token individual de Kaggle que no puede versionarse, y sin él el pipeline quedaría sin ejecutar ni verificar.

Ninguna cifra de este informe describe el fraude real. Cada tabla y figura lleva el rótulo de origen, y cada artefacto registra `data_source: sintetico_sustituto` en su manifest. Los mismos comandos, con los CSV reales, producen resultados reales: la única diferencia es el archivo de entrada.

Página 1 · El problema de decisión

No es un problema de predicción

Predecir `isFraud` no resuelve nada por sí solo. El sistema debe **decidir**, para cada transacción, entre **aprobar**, **enviar a revisión** o **bloquear**, y hacerlo bajo cuatro restricciones que cambian por completo qué significa "un buen modelo":

1. **La etiqueta llega 30 días tarde.** Hoy no se sabe si la decisión de hoy fue correcta. Cualquier métrica de error es necesariamente retrospectiva.
2. **La capacidad humana es de 150 revisiones/día.** No es un parámetro a optimizar: es un techo.
3. **Los errores cuestan cosas distintas.** Un falso negativo cuesta el monto de la transacción; un falso positivo cuesta fricción con un cliente legítimo.
4. **La distribución cambia.** Un modelo entrenado hoy se degrada, y no se entera hasta un mes después.

Objetivos y qué los verifica

Obj.	Enunciado	Criterio	Resultado
O1	Menor costo que la política actual y que el estático	$< 3,59$ y $< 1,96$ UM/tx	Cumplido: 1,28 UM/tx
O2	Limitar la caída de PR-AUC	$\text{Caída} > 10\% \Rightarrow$ alerta	Alerta activa para S0
O3	Respetar la capacidad de revisión	$\leq 150/\text{día}$	Cumplido: nunca excedido en 62 días
O4	Señales sin etiqueta que anticipen la caída	Medir el retraso	KS/PSI y S1 el mismo día; ADWIN a 30 días
O5	Limitar la disparidad entre segmentos	$\text{Brecha} > 2\text{ pp} \Rightarrow$ alerta	13 segmentos con soporte, 0 alertas

Tres familias de métricas

Técnicas — AP (implementado como Average Precision, sin interpolación trapezoidal), $\text{skill} = (\text{AP} - \text{prevalencia}) / (1 - \text{prevalencia})$, F1 y *balanced accuracy* en un umbral declarado, recall a $\text{FPR} \leq 1\%$, recall a precisión $\geq 80\%$, Brier, ECE y latencia p50/p95/p99.

De decisión — costo observado simulado por transacción, monto de fraude evitado, costo de falsos positivos, retorno por analista-hora, revisiones/día y demanda excedente.

Sociales — bloqueo de clientes legítimos (distinto del FPR del clasificador: incluye las legítimas que el analista bloquea por error), disparidad por segmento con soporte e intervalos, y cobertura automática.

Drift de datos frente a drift condicional. Son cosas distintas y el sistema las mide por separado: KS/PSI y el *domain classifier* detectan cambios en **P(X)**; solo ADWIN sobre el error real observa cambios en **P(y|X)**, y por eso llega con 30 días de retraso.

Olvido por ventanas fijas frente a referencias estáticas

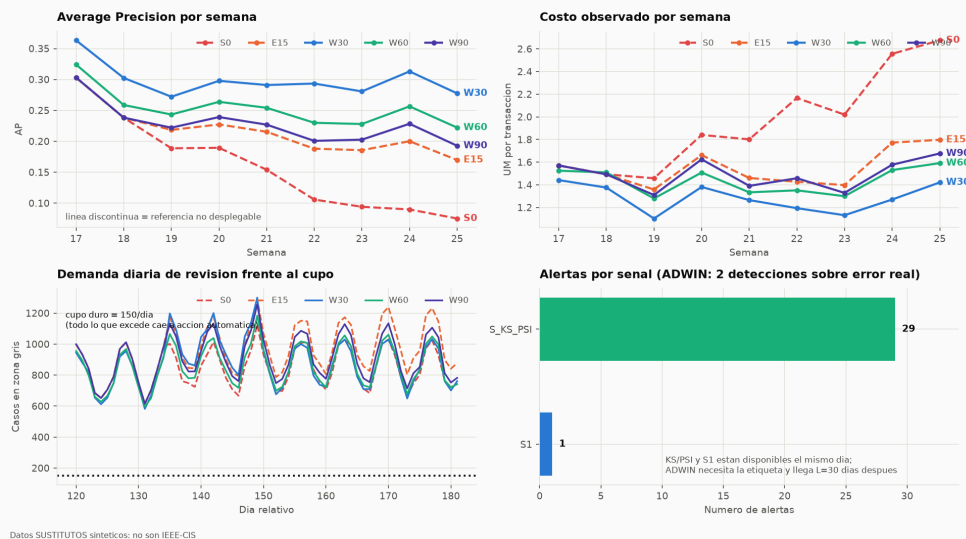


Figura 1 — Ver `docs/arquitectura.mmd` para el diagrama completo de los cinco componentes.

Los cinco componentes

Adquisición e integración · dos fuentes tabulares unidas por `TransactionID` con un `left join` uno-a-uno validado. **Predictor** · logística, Random Forest y LightGBM, con el preprocesamiento ajustado dentro de cada fit. **Decisión** · política de costo esperado con dos umbrales y cupo. **Incertidumbre** · calibración de Platt por versión, intervalos por bootstrap de bloques. **Acción** · tres acciones simuladas, con cola priorizada y apelación documentada.

El cálculo que toma la decisión

Para probabilidad calibrada p y monto m :

$$\begin{aligned} E[\text{aprobar}] &= p \cdot m && (\text{si es fraude, se pierde el monto}) \\ E[\text{bloquear}] &= (1 - p) \cdot c_{FP} && (\text{si era legítima, fricción}) \\ E[\text{revisar}] &= c_R + p \cdot (1 - r_H) \cdot m + (1 - p) \cdot f_H \cdot c_{FP} && (\text{el analista falla a veces}) \end{aligned}$$

con $c_{FP} = 5 \text{ UM}$, $c_R = 1 \text{ UM}$, $r_H = 0,90$, $f_H = 0,02$. UM es consistente con `TransactionAmt`; no se convierte a moneda.

Revisar no siempre gana. Con montos bajos el costo fijo c_R supera la pérdida esperada, y la política prefiere una acción automática. Por eso existe una zona gris y no un umbral único.

Por qué calibrar no es un adorno

Esa aritmética solo tiene sentido si p es una probabilidad. Un modelo con `scale_pos_weight` produce puntajes inflados; usarlos como probabilidad haría bloquear de más **sin que el AP lo notara**. El Platt por versión redujo el ECE de 0,0046 a 0,0010 en LightGBM, de 0,0291 a 0,0014 en Random Forest y de 0,0052 a 0,0015 en la logística.

La cola, sin ventajas irreales

El cupo se reserva **al admitir**, de forma irrevocable y por `event_id`. No se ordena el día completo por $p \times \text{monto}$ para quedarse con los 150 mejores: eso exigiría conocer transacciones que aún no han ocurrido. La prioridad ordena el **servicio** entre los ya admitidos; agotado el cupo, el caso cae a la más barata entre aprobar y bloquear.

Supervisión humana

Ninguna alerta despliega por sí sola. Reentrenar y promover exigen autorización humana registrada, incluso cuando todos los gates técnicos pasan. Sin versión válida, el servicio responde `model_unavailable` (HTTP 503) y pausa: **no** decide el pago por defecto ni degrada a una referencia no desplegable.

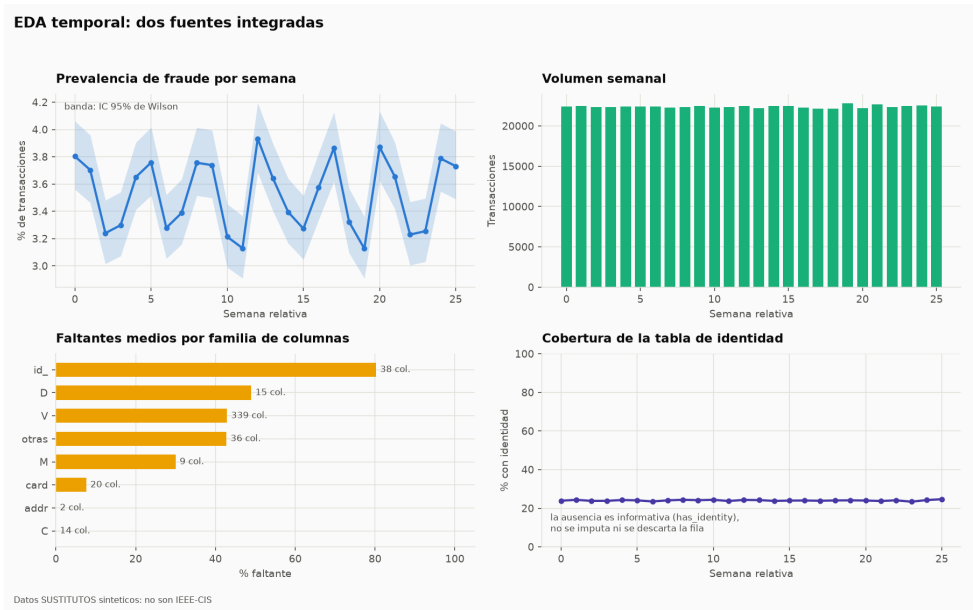


Figura 2 — Prevalencia semanal con IC de Wilson, volumen, faltantes por familia y cobertura de identidad. Datos sustitutos.

Integración auditada

Comprobación	Resultado
Transacciones	581 972 filas × 394 columnas
Identidad	139 843 filas × 41 columnas
Filas tras el join	581 972 (invariante)
Cobertura de identidad	24,0 %
IDs de identidad huérfanos	0
Prevalencia global	3,52 %

La ausencia de identidad **no es un faltante a imputar**: es información (`has_identity`). Descartar esas filas sesgaría el panel hacia los clientes con dispositivo identificado.

Dos relojes, nunca mezclados

`TransactionDT` es un **delta en segundos** desde un origen desconocido, no una fecha. De él se derivan `día`, `semana` y una `hora relativa`, que se usa como ciclo de 24 h y no identifica hora local ni día laboral.

- **Reloj de evento**: cuándo ocurrió la transacción. Gobierna qué features son visibles.
- **Reloj de disponibilidad**: `available_at = día + 30`. Gobierna cuándo una etiqueta puede entrar a un fit, a una métrica o a ADWIN.

Features causales

Se construyen dos proxies de entidad (tarjeta: `card1..card6 + addr1`; dispositivo: `DeviceType + DeviceInfo`) y, para cada uno, conteos y montos en ventanas de 1 h, 24 h y 7 d, rezago del monto, tiempo desde el último evento y conteo expansivo.

Un proxy no es una persona. Se mide su calidad en vez de asumirla: 10 613 entidades de tarjeta, 54,8 eventos por entidad, 19,2 % de *singletons*.

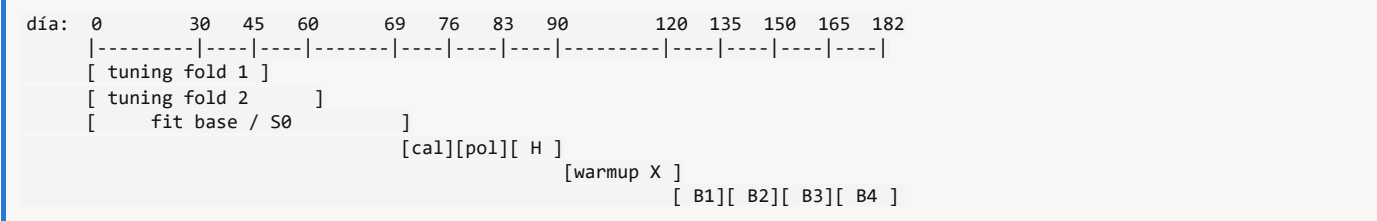
Lo que hace confiable la causalidad

Las features se emiten **antes** de actualizar el estado. El caso que rompe una implementación ingenua son los empates de `TransactionDT`: si dos transacciones comparten instante, procesarlas en secuencia haría que la segunda viera a la primera. Aquí el grupo completo lee el mismo pasado y la incorporación al historial sigue un orden canónico por identificador.

Esto no es una intención documentada: doce pruebas automáticas lo verifican.

Riesgo	Prueba que lo detectaría
La fila actual entra en su propio agregado	<code>test_la_fila_actual_nunca_entra_en_su_propio_agregado</code>
Un evento futuro altera una fila anterior	<code>test_un_evento_futuro_no_altera_features_anteriores</code>
El orden de los empatados cambia el valor	<code>test_permutar_ids_empatados_no_cambia_las_features</code>
El preprocesamiento se ajusta con datos futuros	<code>test_preprocesamiento_no_cambia_al_alterar_filas_futuras</code>
Una etiqueta inmadura influye en el modelo	<code>test_mutar_etiquetas_inmaduras_no_cambia_el_predictor</code>
Dos roles comparten filas	<code>test_roles_reales_no_comparten_ids</code>

Particiones (intervalos semiabiertos, días relativos)



En cada actualización $T \in \{120, 135, 150, 165\}$, con $c = T - 30$:

Rol	Intervalo	Días efectivos
Predictor	$[c-W, c-21)$	9 (W30) / 39 (W60) / 69 (W90)
Calibrador	$[c-21, c-14)$	7
Reserva de política	$[c-14, c-7)$	7
Validación de promoción (H)	$[c-7, c)$	7

Las tres reservas son idénticas para todas las estrategias. Eso es lo que aísla el efecto del tamaño de ventana: si W30 tuviera una cola de calibración distinta de W90, la diferencia de costo dejaría de ser atribuible al olvido.

Una distinción que rompe el pipeline si se confunde

$cutoff = T - L$ delimita qué días de evento pueden usarse. $job_time = T$ es cuándo corre el ajuste, y es contra lo que se compara $available_at$. Con $T = 120$ y $L = 30$, el último día de evento con etiqueta confirmada es el 89. Comparar contra $cutoff$ en lugar de contra job_time exigiría $día < 60$ y dejaría vacías las colas de calibración, política y validación.

Orden de construcción, no intercambiable

$predictor \rightarrow calibrador \rightarrow política \rightarrow validación$. Cada paso usa scores producidos por el anterior sobre datos que ese paso no vio. Invertir el orden haría que el calibrador corrigiera un modelo distinto del que se despliega.

Si un rol no alcanza el soporte mínimo (fit: 200 fraudes / 2 000 legítimas), la versión se marca **no válida**. No se amplía la ventana ni se usan etiquetas inmaduras: eso cambiaría en silencio la W que se está midiendo.

Prerregistro

La familia, los hiperparámetros y los umbrales se congelan usando **solo desarrollo**, y su hash (c6288e32141507c6...) se sella antes de abrir el test. Si la configuración cambiara entre la selección y el test, la corrida se detiene.

Exposición exploratoria declarada. La selección de IEEE-CIS en el benchmark histórico ya examinó periodos tardíos del dataset. Esto sigue siendo un **backtest retrospectivo**, no una validación prospectiva.

Sistema estatico S0 en el periodo de test

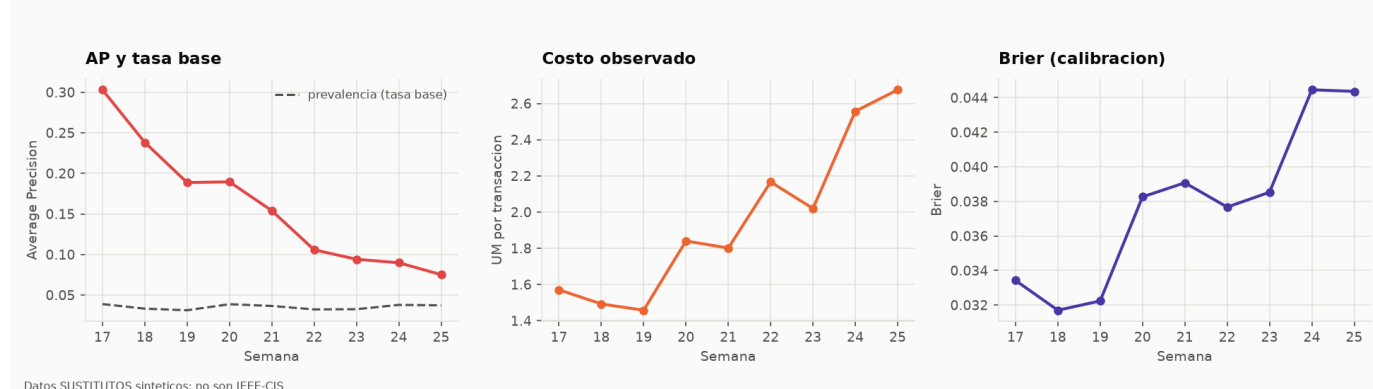


Figura 3 — El AP de S0 cae mientras la tasa base se mantiene plana. Esa es la evidencia de que la pérdida es del modelo, no aritmética.

Comparación de las tres familias (desarrollo)

Selección por **costo**, no por AP. 18 fits de tuning (3 familias × 3 configuraciones × 2 folds forward), luego 3 ajustes finales.

Familia	Config	AP	Costo (UM/tx)	Brier crudo → calibrado	Umbral (τ_bajo, τ_alto)
LightGBM	lgbm_15	0,5083	0,5945	0,02123 → 0,02111	(0,0120 · 0,7740)
Logística	lr_c1	0,4547	0,6606	0,02251 → 0,02218	(0,0149 · 0,7485)
Random Forest	rf_100_12	0,2746	1,0135	0,02950 → 0,02655	(0,0095 · 0,8401)

Referencias simuladas: aprobar todo = 2,73 UM/tx; bloquear todo = 4,85 UM/tx (sobre la reserva de política).

Los umbrales **no son 0,5**: se eligen minimizando el costo observado sobre [76,83], recorriendo pares de cuantiles del score e incluyendo los bordes degenerados. LightGBM se congela como familia adaptativa.

La degradación, leída con cuidado

El AP de S0 cae de 0,258 (B1) a 0,082 (B4) y su costo sube de 1,57 a 2,67 UM/tx. Lo que convierte eso en evidencia útil es que **la prevalencia se mantiene plana** (~3,5 % en los cuatro bloques): si hubiera caído, el AP bajaría por aritmética y no habría nada que concluir sobre el modelo.

La comparación temporal por sí sola **no identifica causalmente** el drift. Lo que se afirma es que el comportamiento es consistente con un cambio en $P(y|X)$, no que se haya probado.

Página 6 · Adaptación por olvido: el experimento central

Cinco estrategias, **los mismos eventos**, la misma semilla, la misma política y la misma familia. 198 587 eventos evaluados por estrategia en los cuatro bloques.

Estrategia	W / cadencia	AP	Skill	Brier	Costo UM/tx [IC 95 %]	Δcosto vs S0 [IC 95 %]	Bloqueo legít.	Versiones
W30	30 d / 15 d	0,2955	0,2697	0,0311	1,2833 [1,199 · 1,359]	-0,676 [-0,999 · -0,424]	7,45 %	4
W60	60 d / 15 d	0,2503	0,2228	0,0333	1,4341 [1,363 · 1,504]	-0,526 [-0,807 · -0,284]	7,68 %	4
W90	90 d / 15 d	0,2247	0,1963	0,0340	1,4899 [1,413 · 1,568]	-0,470 [-0,734 · -0,239]	8,16 %	4
E15 (ref.)	expansiva / 15 d	0,2130	0,1841	0,0342	1,5477 [1,460 · 1,654]	-0,412 [-0,645 · -0,195]	8,53 %	4
S0 (ref.)	fijo / ninguna	0,1445	0,1132	0,0378	1,9596 [1,699 · 2,279]	—	7,92 %	1

Costo de aprobar todo: 3,5912 UM/tx. Intervalos por bootstrap pareado de bloques contiguos de 7 días, 200 remuestreos. S0 y E15 son referencias no desplegables.

AP por bloque: dónde se ve el olvido

Estrategia	B1	B2	B3	B4
W30	0,3189	0,2817	0,2890	0,3001
W60	0,2774	0,2502	0,2347	0,2449
W90	0,2576	0,2268	0,2063	0,2146
E15	0,2576	0,2174	0,1951	0,1889
S0	0,2576	0,1795	0,1132	0,0824

W30 **sostiene** su rendimiento (0,319 → 0,300) mientras S0 pierde dos tercios del suyo. El orden W30 > W60 > W90 > E15 > S0 es monótono en AP y en costo: bajo drift, la frescura vale más que el volumen de datos.

Lo que el sistema NO consiguió

Los umbrales diagnósticos se eligieron en validación y se aplicaron **congelados**. En test no se sostienen:

Estrategia	Recall @ FPR 1 %	FPR real	Recall @ prec. 80 %	Precisión real
W30	0,2948	1,85 %	0,1508	48,9 %
S0	0,1610	2,12 %	0,0788	29,7 %

Ninguna estrategia alcanza el $FPR \leq 1 \%$ ni la precisión $\geq 80 \%$ en el periodo de test. **Esto se reporta, no se corrige moviendo el umbral:** hacerlo convertiría un objetivo incumplido en un resultado ajustado a posteriori.

Detección y sus dos relojes

ADWIN sobre el Brier individual detectó **2 cambios en S0** y **ninguno en las cuatro estrategias que se reentrenan** — coherente: al refrescarse, su error no deriva lo suficiente. KS/PSI generó 29 alertas y S1 una, ambas disponibles el mismo día; ADWIN necesita la etiqueta y llega 30 días después. Esa distancia es la respuesta empírica a O4.

El detector se validó contra verdad conocida: sobre un stream con salto en la observación 1 000, lo detecta en la 1 055 (retraso 55) sin falsas alarmas, y no se alarma en un stream estacionario. Es validación del **instrumento**.

Sensibilidad

Escenario	c_FP	c_R	Costo UM/tx
bajo	1	0,5	0,7346
base	5	1	1,2833
alto	10	2	1,7040
estrés analista (r_H=0,80, f_H=0,05)	5	1	1,3209

La conclusión es robusta al supuesto del analista (+2,9 %) y escala con los costos sin cambiar el ordenamiento entre estrategias.

Equidad operativa

13 segmentos con soporte suficiente ($\geq 1\,000$ legítimas), **0 alertas de brecha**. La mayor diferencia es +0,92 pp (anonymous.com), por debajo del umbral de 2 pp. Son variables de **negocio**: IEEE-CIS no tiene atributos protegidos verificables, así que esto **no es una auditoría demográfica**.

Página 7 · Despliegue, operación y costo

Lo entregado frente a lo diseñado

Entregado y verificado localmente	Diseñado, ejecuta el equipo a mano
Paquete versionado con hashes	<code>docker build / push</code> a Artifact Registry
Servicio HTTP (<code>/health</code> , <code>/predict</code>)	Crear el servicio en Cloud Run
Replay con ledger SQLite transaccional	Apuntar el replay al endpoint remoto
Runbook y política de promoción	Promover, revertir y cerrar recursos

Pub/Sub, Firestore, BigQuery, Cloud Scheduler y Vertex AI Pipelines aparecen en el diagrama como **diseño futuro**: no hay recursos creados ni código de integración.

Evidencia de la demo local

Comprobación	Resultado
Tres acciones sobre datos reales	aprobar 4 294 · bloquear 406 · revisar 300
Cupo respetado	150/150, nunca excedido
Idempotencia (50 reenvíos)	Aprobada: 0 cupos nuevos, cupo estable
Latencia de inferencia, p95	66,3 ms (objetivo ≤ 300 ms) ✓
Latencia p50 / p99	45,0 / 71,0 ms
<code>/health</code> informa versión y hash	<code>W30_T165</code> , <code>4f4b3f06...</code>
Rechazo de <code>isFraud</code> en el payload	HTTP 400
Fixtures de contrato	5/5 (rotulados como sintéticos)

El build de la imagen Docker no pudo verificarse: el daemon de Docker no estaba activo en la máquina de desarrollo. El `Dockerfile` se entrega sin comprobar, y construirlo es el primer paso manual del equipo. El servicio **sí** se verificó de forma nativa sobre HTTP real.

Frecuencia, autonomía y gates

Reentrenamiento cada **15 días** con ventana W30, monitoreo KS/PSI diario, S1 por bloque y ADWIN al madurar cada etiqueta. Cinco gates: integridad (bloqueante), costo $\leq$ aprobar todo en el arranque, no inferioridad (costo $\leq 1,01 \times$ champion, $\Delta AP \geq -0,01$, $\Delta Brier \leq 0,005$), social (+2 pp máximo) y **humano, siempre**.

La regla es de **no inferioridad tolerante a ruido**, no de superioridad: exigir mejora en cada ciclo llevaría a no actualizar nunca bajo ruido.

Se separan dos cosas que suelen confundirse. El **rollback técnico** —tasa de error HTTP por encima del 1 %, o p95 por encima de 300 ms en dos lotes consecutivos— es inmediato y preautorizado. El **deterioro de negocio** solo puede afirmarse con etiquetas maduras, 30 días después, y exige revisión humana explícita.

Costo de cómputo

Trabajo	Tareas	Minutos
Tuning	18	9,0
Ajustes finales y de adaptación	10	6,2
Backtest completo	1	5,2
Datos y features	20	1,4
Total		21,9 de 480

Muy por debajo del presupuesto: el perfil reducido de recursos no fue necesario.

Escala, costo e integración

Sin estimación de factura. Los factores: CPU/RAM por petición y cold start en serving; W y ancho del panel en reentrenamiento; versiones retenidas y logs por evento en almacenamiento. Tres límites declarados: el cupo solo está garantizado para un orquestador **secuencial**; la latencia es local y no representa una región cloud; el prototipo asume identidad **simultánea** y features precomputadas.

Página 8 · Límites, riesgos y conclusión

Qué no puede concluirse

Afirmación tentadora	Por qué no se sostiene
"El sistema ahorra un 34,5 % de costo"	Los costos son simulados bajo supuestos declarados. No hay ahorro causal medido sobre pagos reales
"Probamos que hay concept drift"	La comparación temporal es consistente con un cambio en $P(y X)$, no lo identifica causalmente
"W30 es la ventana óptima"	Es la mejor entre las tres evaluadas, con esta cadencia y esta semilla. W14 y otras cadencias no se ejecutaron
"El sistema es justo"	Es disparidad operativa por variables de negocio, sin atributos protegidos verificables
"Funciona en producción"	El cupo está garantizado para un orquestador secuencial; la demo no certifica concurrencia
"Estos son resultados de IEEE-CIS"	Son de un sustituto sintético ; los comandos son idénticos, los datos no

Selective labels

El núcleo usa **full-information**: toda etiqueta madura a los 30 días incluso si la acción simulada bloqueó. En operación real una transacción bloqueada nunca revela su desenlace, así que las etiquetas disponibles estarían sesgadas por las propias decisiones del sistema. Es una limitación **no resuelta experimentalmente** aquí, y la corrección con *masked-label* quedó fuera del alcance.

Una característica del diseño que vale la pena señalar

La zona gris se define **solo sobre p** . Una transacción de monto cero con p intermedia consume un cupo de revisión, aunque aprobarla cueste exactamente 0. El monto solo interviene en la prioridad de servicio y en el overflow. La vía de mejora es directa —admitir a revisión únicamente si $E[\text{revisar}] < \min(E[\text{aprobar}], E[\text{bloquear}])$ — pero desviaría del diseño de dos umbrales globales especificado, así que se documenta en lugar de aplicarse.

La restricción que más limita el sistema

La demanda de revisión es de **600–1 300 casos/día** frente a un cupo de 150: una sobresuscripción de 5 a 8×. El cupo, no el modelo, es el cuello de botella dominante. Mejorar el AP tiene rendimientos decrecientes mientras el 90 % de la zona gris caiga a una acción automática por falta de capacidad.

Riesgos y controles

Riesgo	Control entregado
Scores descalibrados	Platt por versión con cola separada; Brier y ECE reportados
Oscilación de versiones	Cadencia y cooldown de 15 días; sin retuning por alertas
Olvido de patrones raros	Comparación de 9 / 39 / 69 días de fit y por segmento
Etiquetas tardías	<code>available_at</code> y predicciones inmutables
Decisiones sin modelo válido	503 y pausa; nunca una acción por defecto
Confundir un bug con drift	Anomalías e integridad se revisan antes de atribuir
Retuning tras ver el test	Hash de prerregistro verificado; la corrida aborta

Conclusión

Bajo el drift inyectado, **olvidar compensa y compensa más cuanto más corta es la ventana**: W30 reduce el costo observado un 34,5 % frente al modelo estático y un 64,3 % frente a aprobar todo, con un intervalo pareado que no cruza el cero ($-0,676$ [$-0,999, -0,424$] UM/tx). El orden $W30 > W60 > W90 > E15 > S0$ es monótono en AP y en costo, y W30 sostiene su rendimiento en los cuatro bloques mientras el estático pierde dos tercios del suyo.

Al mismo tiempo, **ningún método alcanzó los objetivos diagnósticos** de $FPR \leq 1\%$ ni de precisión $\geq 80\%$ en el periodo de test con los umbrales congelados, y la capacidad de revisión resultó ser una restricción más limitante que la calidad del modelo.

La recomendación operativa es la ventana elegida **en desarrollo**, con reentrenamiento cada 15 días y aprobación humana por acto. La validación futura requiere datos reales de IEEE-CIS y, sobre todo, una evaluación prospectiva: esto es un backtest, y un backtest no prueba que mañana funcione.

Referencias y trazabilidad

Cada cifra de este informe es trazable a un `run_id` y a un artefacto con hash en `reports/indice_evidencia.csv`.

- **Antecedentes:** `propuesta_proyecto1_final.md` (avance 3.1), `concept_drift_findings.md` y `concept_drift_benchmark_instructions.md` (selección del dataset; citados como evidencia consistente con drift, **no** como prueba causal, y con las limitaciones de preprocesamiento de su benchmark declaradas).
- **Protocolo:** `docs/protocolo_experimental.md` · **Contrato:** `docs/contrato_sistema.md` · **Reproducción:** `docs/reproducibilidad.md`
- **Despliegue:** `deploy/gcp_runbook.md` y `deploy/promocion_rollback.md`
- **Contrato de Cloud Run:** documentación oficial de Google Cloud (container contract y despliegue de imágenes).
- **Evidencia ejecutable:** tres notebooks en `notebooks/`, 127 pruebas en `tests/`.